# RTOS HW1：Hello uCOS-II

作業報告、核心紀錄與實際執行驗證

## 學號：M11507W14

完成日期：2026-10-11

## 一、作業目標與實作範圍

兩個任務在各自的主迴圈輸出 Task(ID) is running，之後呼叫延遲函式。

任務 1：優先權 1、延遲 4 ticks；任務 2：優先權 2、延遲 7 ticks。

在核心的啟動、任務排程、中斷排程與就緒表更新位置加入觀測程式。

保持原有 Micrium / Microsoft 目錄、OS2 / OS3 專案及 Win32 port。

不改變 OS_SchedNew 的位元表排程演算法，也不加入額外排程任務。

## 二、修改檔案

OS2/main.c：兩個任務、堆疊、優先權與 OSTimeSet(0)。

OS2/app_hooks.c：輸入驗證、任務標籤、共用輸出與紀錄格式。

uCOS-II/Source/ucos_ii.h：extern 宣告與紀錄函式原型。

uCOS-II/Source/os_core.c：初次派送、真正切換、延遲到期的 ready 紀錄。

uCOS-II/Source/os_time.c：延遲任務的 waiting 紀錄。

## 三、實際結果

Debug / Win32 已成功編譯並執行，保留原本的 30 tick 模擬上限。

Task 1 執行時刻：0、4、8、12、16、20、24、28。

Task 2 執行時刻：0、7、14、21、28。

tick 28：兩個任務同時就緒，優先權 1 先執行，再執行優先權 2。

61 行實際紀錄；初次派送計數為 0，真正 context switch 共 23 次。

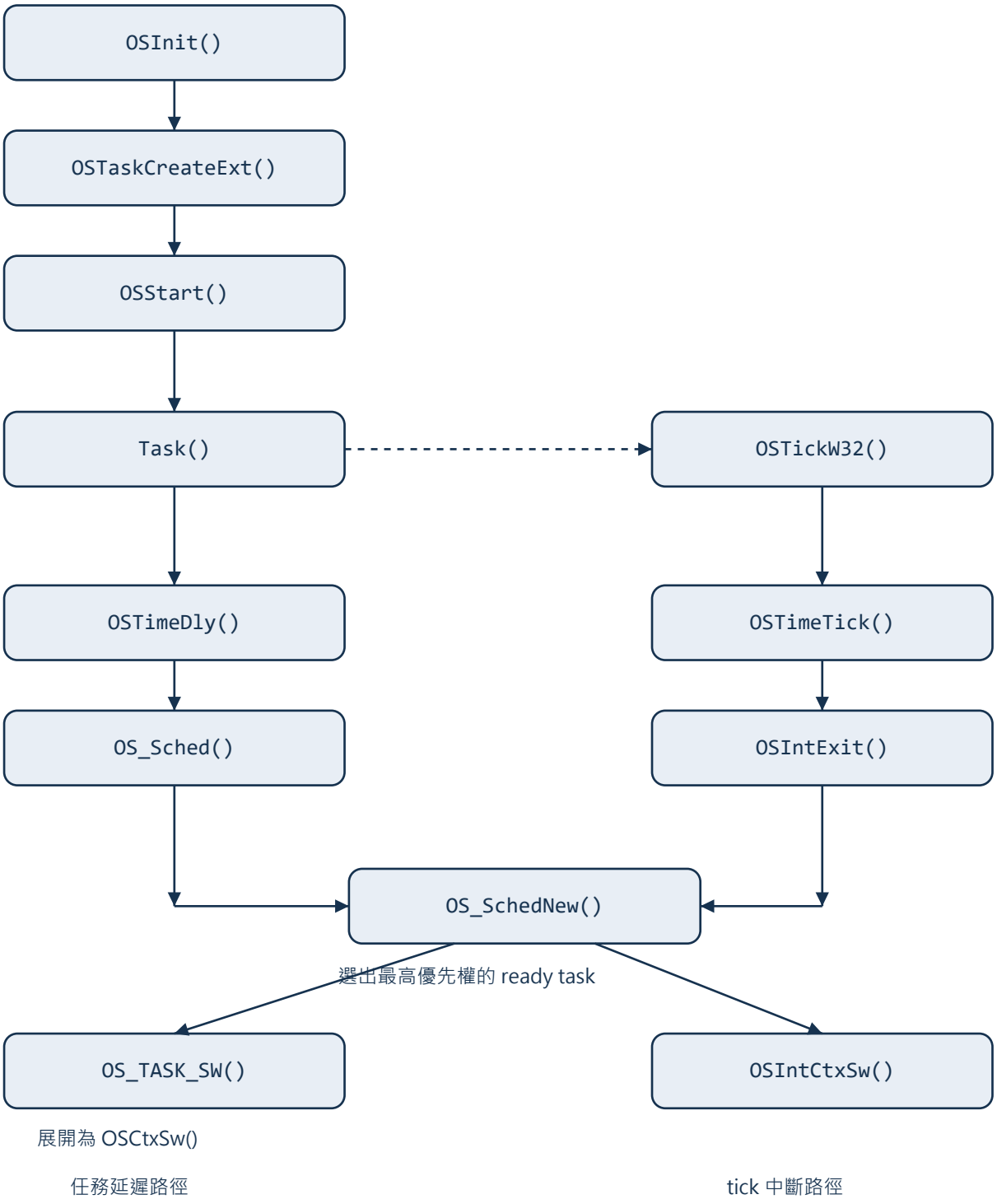
### 輸入與輸出：./TaskSet.txt、./Output.txt

TaskSet 的 ExecutionTime 欄位保留為資料；HW1 要求印訊息後延遲，

因此不將此欄位解讀為額外 CPU busy-wait 或新的排程需求。

# 系統流程圖

依作業圖填入函式，並補充真正的 Win32 context switch 路徑




初次啟動：OSStart 內先呼叫 OS_SchedNew，再由 OSStartHighRdy 啟動任務。

兩種後續切換都保留 Win32 port 的 OSTaskSwHook 與 TCB 更新機制。

Task 延遲後執行下一個 ready task；延遲到期由 tick 路徑重新派送任務。

沒有應用任務就緒時，OS_TaskIdle（優先權 63）執行。

# 函式與就緒位元表

數字越小優先權越高；紀錄觀察原本的排程流程

## OSInit

初始化核心、TCB 與 ready table，並建立 idle task。

## OSTaskCreateExt

建立 TCB、設定入口與堆疊；ExtPtr 指向任務參數。

## OSTimeSet / OSTimeGet

啟動前設為 0；紀錄時讀取核心 tick。

## OSStart / OSStartHighRdy

選擇第一個 ready task，讓 Win32 任務執行緒開始執行。

## OSTimeDly

移除目前任務的 ready bit，設定 OSTCBDly，再呼叫 OS_Sched。

## OS_Sched

任務層排程；有不同的最高就緒任務時增加計數並切換。

## OSTickW32 / OSIntEnter

Win32 tick 執行緒暫停目前任務，進入模擬中斷。

## OSTimeTick

增加 OSTime、遞減延遲；延遲到期時恢復 ready bit。

## OSIntExit

中斷結束後選擇 ready task，需要時呼叫 OSIntCtxSw。

## OS_SchedNew

以 OSUnMapTbl 查找 OSRdyGrp 與 OSRdyTbl 中最低有效位元。

## OSCtxSw / OSIntCtxSw

由原有 Win32 port 切換 TCB 與任務執行緒。

## Ready bits：prio 1 = 0x02，prio 2 = 0x04

OSRdyTbl[0] 的兩個任務位元都就緒時為 0x06；清除任務 1 後為 0x04。

任務 2 再延遲後為 0x00，OSRdyGrp 的 group 0 位元清除。

idle 在 group 7，因此 OSRdyGrp 仍有 0x80，不會變成全零。

紀錄的是該任務所屬的 OSRdyTbl[OSTCBY]，不是整個 ready table。

# 修改截圖：任務與共用宣告

由本次實際原始碼擷取；黃色區塊標示相對原始提交的修改

main.c | lines 64-103 | yellow = modified

```
64  int main(void)
65  {
66      INT8U err;
67      int n;
68
69      CPU_IntInit();
70      Mem_Init();
71      CPU_IntDis();
72      CPU_Init();
73      OSInit();
74      InputFile();
75      OutFileInit();
76
77      /* HW1: priorities are 1 and 2; job numbers start at zero. */
78      for (n = 0; n < TASK_NUMBER; ++n) {
79          err = OSTaskCreateExt(n == 0 ? task1 : task2,
80                                &TaskParameter[n],
81                                &TaskStacks[n][TASK_STACKSIZE - 1u],
82                                (INT8U)TaskParameter[n].TaskPriority,
83                                TaskParameter[n].TaskID,
84                                &TaskStacks[n][0], TASK_STACKSIZE,
85                                &TaskParameter[n],
86                                OS_TASK_OPT_STK_CHK | OS_TASK_OPT_STK_CLR);
87          if (err != OS_ERR_NONE) {
88              fprintf(stderr, "OSTaskCreateExt failed: %u\n", (unsigned)err);
89              fclose(Output_fp);
90              return EXIT_FAILURE;
91          }
92      }
93      /* HW1: the initial dispatch is at tick zero. */
94      OSTimeSet(0u);
95      OSStart();
96      return EXIT_FAILURE;  /* OSStart should not return. */
97  }
98
99  /* HW1: execution time in TaskSet is metadata, not a busy-wait for HW1. */
100 static void TaskBody(void *p_arg)
101 {
102     task_para_set *task_data = (task_para_set *)p_arg;
103     OS_CPU_SR cpu_sr = 0u;
```

main.c | lines 100-113 | yellow = modified

```
100  static void TaskBody(void *p_arg)
101  {
102      task_para_set *task_data = (task_para_set *)p_arg;
103      OS_CPU_SR cpu_sr = 0u;
104
105      while (1) {
106          OS_ENTER_CRITICAL();
107          HW1_Log("%lu\tTask(%u) is running\n",
108                  (unsigned long)OSTimeGet(), (unsigned)task_data->TaskID);
109          OS_EXIT_CRITICAL();
110          OSTimeDly(task_data->TaskPeriodic);
111      }
112  }
113
```

每個任務以自己的參數呼叫 OSTimeDly，沒有改動 OS2 專案參照。

ucos_ii.h 的共用變數改為 extern，定義集中於既有 app_hooks.c。

這避免各個 C 編譯單元重複定義；job 初值 0，優先權為 1 與 2。

# 修改截圖：核心 context switch

核心層新增觀測點；切換與計數仍由原本 uCOS-II 執行

os_core.c | lines 873-886 | yellow = modified

```
873  void  OSStart (void)
874  {
875      if (OSRunning == OS_FALSE) {
876          OS_SchedNew();  /* ... */
877          OSPrioCur     = OSPrioHighRdy;
878          OSTCBHighRdy  = OSTCBPrioTbl[OSPrioHighRdy];  /* ... */
879          OSTCBCur      = OSTCBHighRdy;
880          /* HW1: initial dispatch, not a context switch (counter is zero). */
881          HW1_TraceSwitch(1u);
882          OSStartHighRdy();  /* ... */
883      }
884  }
885
886
```

OSStart：********** 代表尚無 outgoing task，初次派送不增加計數。

os_core.c | lines 1736-1759 | yellow = modified

```
1736              if (OSPrioHighRdy != OSPrioCur) {  /* ... */
1737  #if OS_TASK_PROFILE_EN > 0u
1738                  OSTCBHighRdy->OSTCBCtxSwCtr++;  /* ... */
1739  #endif
1740                  OSCtxSwCtr++;  /* ... */
1741
1742  #if OS_TASK_CREATE_EXT_EN > 0u
1743  #if defined(OS_TLS_TBL_SIZE) && (OS_TLS_TBL_SIZE > 0u)
1744                  OS_TLS_TaskSw();
1745  #endif
1746  #endif
1747
1748                  /* HW1: counter already incremented; log before switching TCBs. */
1749                  HW1_TraceSwitch(0u);
1750                  OS_TASK_SW();  /* ... */
1751              }
1752          }
1753      }
1754      OS_EXIT_CRITICAL();
1755  }
1756
1757
1758  /*
1759  ************************************************************************************************************
```

OS_Sched：在原本的 OSCtxSwCtr++ 之後、OS_TASK_SW 前紀錄。

排程條件、TCB 選擇、原有計數與 context switch 皆維持原樣。

只有在 OSPrioHighRdy 與 OSPrioCur 不同時，才記錄真正的切換。

OSStart 的初次派送則獨立列出，不將其算成一次切換。

# 修改截圖：中斷層排程

tick 喚醒任務時，必須觀察 OSIntExit 而不只是 OS_Sched

os_core.c | lines 710-728 | yellow = modified

```
710                     OS_SchedNew();
711                     OSTCBHighRdy = OSTCBPrioTbl[OSPrioHighRdy];
712                     if (OSPrioHighRdy != OSPrioCur) {  /* ... */
713   #if OS_TASK_PROFILE_EN > 0u
714                         OSTCBHighRdy->OSTCBCtxSwCtr++;  /* ... */
715   #endif
716                         OSCtxSwCtr++;  /* ... */
717
718   #if OS_TASK_CREATE_EXT_EN > 0u
719   #if defined(OS_TLS_TBL_SIZE) && (OS_TLS_TBL_SIZE > 0u)
720                         OS_TLS_TaskSw();
721   #endif
722   #endif
723                         OS_TRACE_ISR_EXIT_TO_SCHEDULER();
724
725                         /* HW1: counter already incremented; log before switching TCBs. */
726                         HW1_TraceSwitch(0u);
727                         OSIntCtxSw();  /* ... */
728                     } else {
```

OSIntExit：涵蓋 tick 喚醒造成的中斷層切換，避免只記到任務延遲。

同一個切換只由其呼叫路徑記錄一次，App_TaskSwHook 不再另加紀錄。

原有 OSCtxSwCtr++ 在這個觀測點之前，所以記錄的是更新後的計數。

tick 執行緒暫停目前任務，進入 OSIntEnter，再呼叫 OSTimeTick。

所有延遲任務的 ready bits 更新完後，OSIntExit 才進行優先權選擇。

因此 tick 28 即使 TCB 串列先處理 task 2，仍會先派送 task 1。

OSIntCtxSw 仍由原有 Win32 port 切換至被選出的任務執行緒。

# 修改截圖：waiting 與 ready

前後值在既有 critical section 中擷取，避免觀測到不一致的位元表

os_time.c | lines 71-90 | yellow = modified

```
71      if (ticks > 0u) {  /* ... */
72          OS_ENTER_CRITICAL();
73          y            =  OSTCBCur->OSTCBY;  /* ... */
74          /* HW1 BEGIN: snapshot ready bits before putting the task to sleep. */
75          hw1_group_before = OSRdyGrp;
76          hw1_table_before = OSRdyTbl[y];
77          OSRdyTbl[y] &= (OS_PRIO)~OSTCBCur->OSTCBBitX;
78          OS_TRACE_TASK_SUSPENDED(OSTCBCur);
79          if (OSRdyTbl[y] == 0u) {
80              OSRdyGrp &= (OS_PRIO)~OSTCBCur->OSTCBBitY;
81          }
82          HW1_TraceReady("waiting", OSTCBCur, hw1_group_before, hw1_table_before);
83          /* HW1 END */
84          OSTCBCur->OSTCBDly = ticks;  /* ... */
85          OS_TRACE_TASK_DLY(ticks);
86          OS_EXIT_CRITICAL();
87          OS_Sched();  /* ... */
88      }
89  }
90
```

OSTimeDly：先保存 group/table，再清除任務 ready bit。

完成位元表修改後輸出 waiting；之後設定延遲並呼叫 OS_Sched。

os_core.c | lines 1009-1029 | yellow = modified

```
1009
1010                    if ((ptcb->OSTCBStat & OS_STAT_SUSPEND) == OS_STAT_RDY) {  /* ... */
1011                        /* HW1 BEGIN: capture only the table byte being changed. */
1012                        hw1_group_before = OSRdyGrp;
1013                        hw1_table_before = OSRdyTbl[ptcb->OSTCBY];
1014                        OSRdyGrp               |= ptcb->OSTCBBitY;  /* ... */
1015                        OSRdyTbl[ptcb->OSTCBY] |= ptcb->OSTCBBitX;
1016                        /* Each completed delay releases the next job. */
1017                        if (ptcb->OSTCBExtPtr != NULL && ptcb->OSTCBPrio <= 2u) {
1018                            ((task_para_set *)ptcb->OSTCBExtPtr)->TaskNumber++;
1019                        }
1020                        HW1_TraceReady("ready", ptcb, hw1_group_before, hw1_table_before);
1021                        /* HW1 END */
1022                        OS_TRACE_TASK_READY(ptcb);
1023                    }
1024                }
1025            }
1026            ptcb = ptcb->OSTCBNext;  /* ... */
1027            OS_EXIT_CRITICAL();
1028        }
1029    }
```

OSTimeTick：只有延遲到期且未 suspend 的任務會被設為 ready。

在恢復 ready bit 後更新 job number，再記錄 ready；所以即將執行的

任務在 context switch 行中已具有新 job number。

# 修改截圖：輸出格式與 task label

沿用既有 app_hooks.c 作為紀錄輔助函式的位置

app_hooks.c | lines 151-199 | yellow = modified

```
151  void HW1_Log(const char *format, ...)
152  {
153      char line[256];
154      va_list args;
155      va_start(args, format);
156      vsnprintf(line, sizeof(line), format, args);
157      va_end(args);
158      printf("%s", line);
159      if (Output_fp != NULL) {
160          fputs(line, Output_fp);
161          fflush(Output_fp);
162      }
163  }
164
165  static void HW1_TaskLabel(OS_TCB *ptcb, char *label, size_t size)
166  {
167      task_para_set *data;
168      if (ptcb->OSTCBPrio == OS_TASK_IDLE_PRIO) {
169          snprintf(label, size, "task(%u)", (unsigned)ptcb->OSTCBPrio);
170      } else {
171          data = (task_para_set *)ptcb->OSTCBExtPtr;
172          snprintf(label, size, "task(%u)(%u)",
173                   (unsigned)data->TaskID, (unsigned)data->TaskNumber);
174      }
175  }
176
177  void HW1_TraceSwitch(INT8U initial)
178  {
179      char current[48], next[48];
180      if (initial != 0u) {
181          strcpy_s(current, sizeof(current), "***********");
182      } else {
183          HW1_TaskLabel(OSTCBCur, current, sizeof(current));
184      }
185      HW1_TaskLabel(OSTCBHighRdy, next, sizeof(next));
186      HW1_Log("%lu\t%s\t%s\t%lu\n", (unsigned long)OSTimeGet(),
187              current, next, (unsigned long)OSCtxSwCtr);
188  }
189
190  void HW1_TraceReady(const char *state, OS_TCB *ptcb,
191                      OS_PRIO group_before, OS_PRIO table_before)
192  {
193      HW1_Log("%lu\t%s\tprio(%u)\tosrdygrp(0x%02X)(0x%02X)"
194              "\tosrdytbl(0x%02X)(0x%02X)\n", (unsigned long)OSTimeGet(),
195              state, (unsigned)ptcb->OSTCBPrio, (unsigned)group_before,
196              (unsigned)OSRdyGrp, (unsigned)table_before,
197              (unsigned)OSRdyTbl[ptcb->OSTCBY]);
198  }
199  /* HW1 END */
```

HW1_Log 先格式化一行，再輸出到主控台與同一個 Output.txt 串流。

每行 fflush，確保 30 tick 模擬結束時輸出已落盤。

一般任務顯示 task(ID)(job)，idle 特別顯示 task(63)。

waiting / ready 的 bitmap 都以 0x 開頭；欄位使用真正的 \t 字元。

核心觀測點及 TaskBody 的紀錄均受原有 critical section 保護。

# 修改截圖：TaskSet 輸入驗證

保留標準輸入檔名；輸入錯誤時不啟動核心，也不覆寫輸出檔案

app_hooks.c | lines 110-156 | yellow = modified

```
110  void InputFile(void)
111  {
112      char line[256];
113      char extra;
114      unsigned id, arrival, execution, period;
115      int n = 0;
116
117      if (fopen_s(&fp, INPUT_FILE_NAME, "r") != 0 || fp == NULL) {
118          fprintf(stderr, "Cannot open %s\n", INPUT_FILE_NAME);
119          exit(EXIT_FAILURE);
120      }
121      while (fgets(line, sizeof(line), fp) != NULL) {
122          if (strspn(line, " \t\r\n") == strlen(line)) {
123              continue;
124          }
125          if (n >= 2 || sscanf_s(line, "%u %u %u %u %c", &id, &arrival,
126                                 &execution, &period, &extra, 1u) != 4 ||
127              id != (unsigned)(n + 1) || arrival != 0u ||
128              execution > 65535u || period != (n == 0 ? 4u : 7u)) {
129              fprintf(stderr, "HW1 requires rows: 1 0 C 4 and 2 0 C 7.\n");
130              fclose(fp);
131              exit(EXIT_FAILURE);
132          }
133          TaskParameter[n].TaskID = (INT16U)id;
134          TaskParameter[n].TaskArriveTime = (INT16U)arrival;
135          TaskParameter[n].TaskExecutionTime = (INT16U)execution;
136          TaskParameter[n].TaskPeriodic = (INT16U)period;
137          TaskParameter[n].TaskPriority = (INT16U)(n + 1);
138          TaskParameter[n].TaskNumber = 0u;
139          ++n;
140      }
141      if (ferror(fp) || n != 2) {
142          fprintf(stderr, "TaskSet must contain exactly two valid tasks.\n");
143          fclose(fp);
144          exit(EXIT_FAILURE);
145      }
146      fclose(fp);
147      fp = NULL;
148      TASK_NUMBER = n;
149  }
150
151  void HW1_Log(const char *format, ...)
152  {
153      char line[256];
154      va_list args;
155      va_start(args, format);
156      vsnprintf(line, sizeof(line), format, args);
```

使用 fgets 成功讀到一行才處理，避免原本 while (!feof(fp)) 的多讀一行。

逐列檢查四個欄位；要求兩個任務、ID 1/2、到達時間 0、延遲 4/7。

不接受缺少任務、多餘任務、錯誤欄位或額外 token。

優先權以 n + 1 設定，避免原始範例中 j 從 0 開始的錯誤。

在 InputFile 成功後才建立 Output.txt，保留錯誤執行前的有效結果。

# 實際輸出與驗證

資料來自編譯後的 OS2.exe，並非離線模擬產生的範例

Output.txt | native Debug / Win32 run

```
0  **********  task(1)(0)  0
0  Task(1) is running
0  waiting  prio(1)  osrdygrp(0x81)(0x81)  osrdytbl(0x06)(0x04)
0  task(1)(0)  task(2)(0)  1
0  Task(2) is running
0  waiting  prio(2)  osrdygrp(0x81)(0x80)  osrdytbl(0x04)(0x00)
0  task(2)(0)  task(63)  2
4  ready  prio(1)  osrdygrp(0x80)(0x81)  osrdytbl(0x00)(0x02)
4  task(63)  task(1)(1)  3
4  Task(1) is running
4  waiting  prio(1)  osrdygrp(0x81)(0x80)  osrdytbl(0x02)(0x00)
4  task(1)(1)  task(63)  4
7  ready  prio(2)  osrdygrp(0x80)(0x81)  osrdytbl(0x00)(0x04)
7  task(63)  task(2)(1)  5
7  Task(2) is running
7  waiting  prio(2)  osrdygrp(0x81)(0x80)  osrdytbl(0x04)(0x00)
...
28  ready  prio(2)  osrdygrp(0x80)(0x81)  osrdytbl(0x00)(0x04)
28  ready  prio(1)  osrdygrp(0x81)(0x81)  osrdytbl(0x04)(0x06)
28  task(63)  task(1)(7)  21
28  Task(1) is running
28  waiting  prio(1)  osrdygrp(0x81)(0x81)  osrdytbl(0x06)(0x04)
28  task(1)(7)  task(2)(4)  22
28  Task(2) is running
28  waiting  prio(2)  osrdygrp(0x81)(0x80)  osrdytbl(0x04)(0x00)
28  task(2)(4)  task(63)  23
```

## 自動核對結果

PASS：兩個任務的實際執行 ticks 分別符合 4 與 7 的間隔。

PASS：所有 waiting / ready 的前後值與獨立位元表計算相符。

PASS：job number、tick 28 優先權順序、0..23 的切換計數連續。

PASS：缺檔、空檔、錯誤欄位、只有一列、第三列均以非零狀態結束。

建置：Visual Studio 2026 / MSVC v145，Debug / Win32，原專案維持 v143。

執行：工作目錄為 OS2/VS，保留 1 tick/秒；tick 31 依原設定正常結束。

## 計數約定與提交內容

初次派送計數 0；第一個真正切換計數 1，採用原有 OSCtxSwCtr。

題目截圖在第一個切換仍顯示 0；此處保留核心真實計數，沒有減一。

ZIP 內含本報告、原專案樹、TaskSet.txt、Output.txt 與 Win32 執行檔。

開啟 OS2.sln，選 Debug / Win32；若使用 v145，依 README 的指令建置。